ESCUELA DE INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

MANUAL TECNICO

PREOYECTO FINAL LFP

Nombre: Bily Estuardo vallecidos folgar

Curso: Lenguajes Formales y Programación

Fecha: 01/05/2026

1. Introducción

La arquitectura y la implementación interna del analizador léxico y sintáctico para el lenguaje de dominio específico TaskScript. El propósito principal de este sistema es procesar archivos de texto plano que definen tableros Kanban, validando estrictamente su estructura gramatical para posteriormente generar reportes estadísticos y visuales. Todo el motor de compilación ha sido desarrollado desde cero en C++ estándar, sin el uso de expresiones regulares nativas (std::regex) ni herramientas de generación automática de parsers, asegurando un control absoluto sobre el procesamiento de la memoria y la evaluación del autómata.

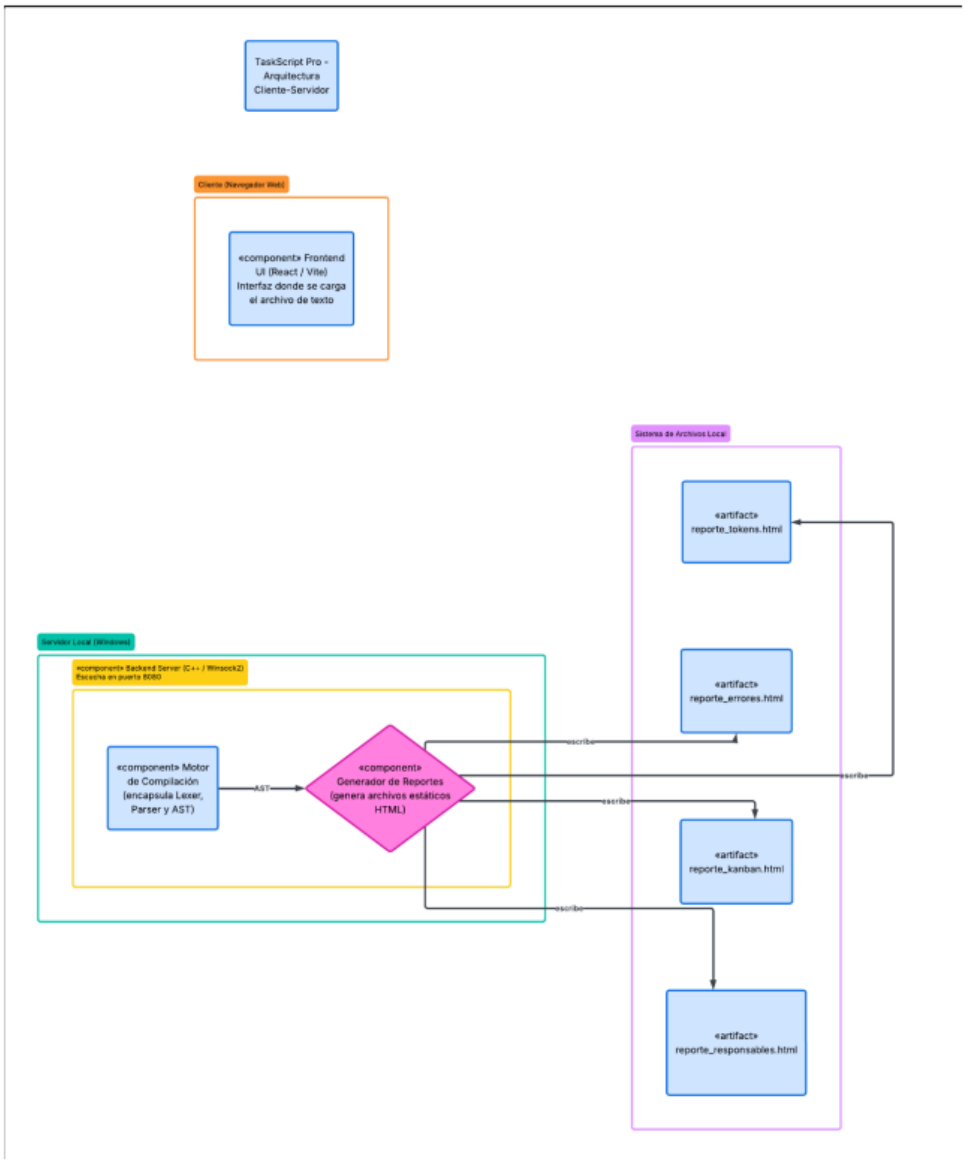
2. Arquitectura del Sistema

El proyecto fue diseñado utilizando un patrón arquitectónico de Cliente-Servidor, logrando una alta cohesión y un bajo acoplamiento entre la interfaz de usuario y el motor de procesamiento lógico.

- Backend (Motor de Compilación): Desarrollado en C++ puro. En lugar de atar el código a una interfaz gráfica nativa pesada, se implementó un servidor local utilizando la librería nativa . Este servidor expone un endpoint mediante el método HTTP POST que recibe el código fuente crudo, lo somete al pipeline de compilación (análisis léxico, sintáctico y semántico básico), escribe físicamente los reportes requeridos en el disco local y retorna un objeto JSON estructurado con los tokens, errores y el código DOT.
- Frontend (Interfaz Gráfica): Desarrollado en React y empaquetado con Vite. Actúa como el cliente que consume el servicio del backend. Esta decisión arquitectónica permite abstraer la complejidad del manejo de archivos y la presentación de datos, delegando el procesamiento computacional pesado a C++ mientras se garantiza una experiencia de usuario fluida, asíncrona y responsiva en el navegador.

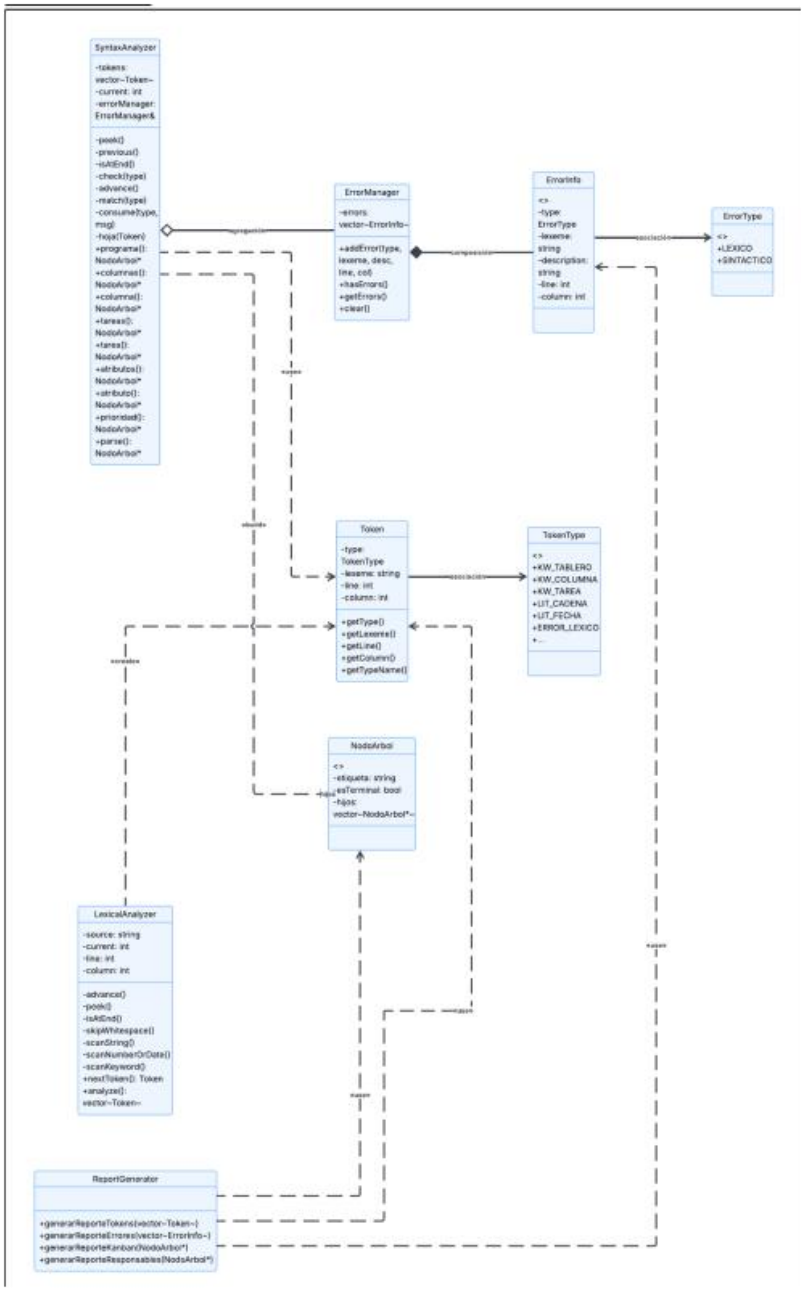
ESCUELA DE INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Diagrama de Componentes (UML)



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Diagrama de Clases



3. Estructura de Clases y Responsabilidades

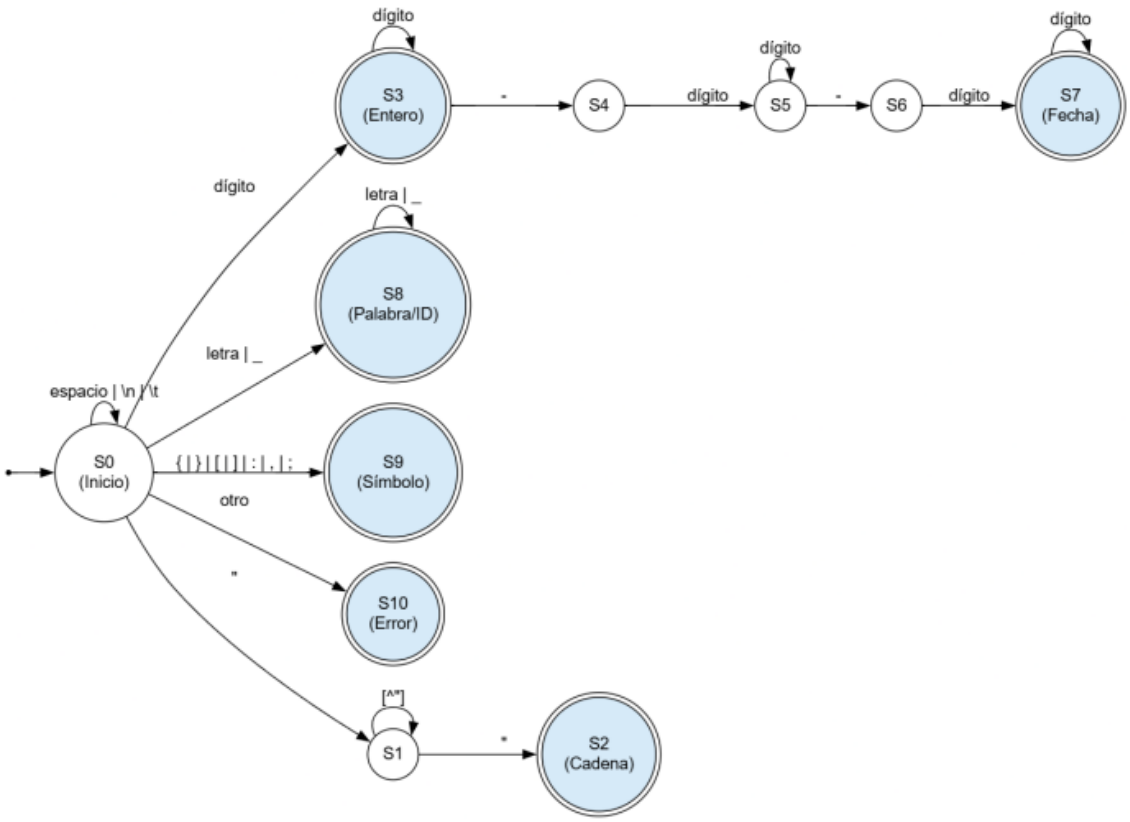
El motor lógico en C++ aplica el Principio de Responsabilidad Única (SRP), dividiendo el flujo de ejecución en las siguientes clases fundamentales:

- Token: Estructura de datos inmutable que encapsula la información de cada unidad léxica, almacenando su lexema original, su tipo (basado en un enum class), y su ubicación espacial exacta (línea y columna) para facilitar el reporte de errores.
- LexicalAnalyzer: Módulo encargado de leer el flujo de caracteres de entrada (string) y agruparlos secuencialmente en tokens válidos basándose en un Autómata Finito Determinista (AFD) explícito.
- SyntaxAnalyzer: Recibe el flujo de tokens y verifica que el orden de estos cumpla con la Gramática Libre de Contexto (GLC). Construye dinámicamente un Árbol de Sintaxis Abstracta (AST).
- NodoArbol: Estructura N-aria que representa los nodos del AST. Contiene un vector de punteros a sus nodos hijos, una etiqueta y un indicador de si es un nodo terminal (hoja) o no terminal.
- ErrorManager: Clase singleton/centralizada que actúa como recolector de incidentes. Almacena objetos ErrorInfo aislando la gestión de errores del resto de la lógica de compilación.
- ReportGenerator: Módulo de salida que realiza un recorrido en preorden sobre el AST para extraer la información de las tareas y columnas, inyectándola en plantillas HTML y CSS predefinidas para generar los reportes del sistema.

4. Análisis Léxico (Autómata Finito Determinista)

El análisis léxico se realiza mediante la técnica de lectura bajo demanda utilizando los métodos advance() y peek(). El autómata fue diseñado para ignorar espacios en blanco y saltos de línea (llevando un conteo preciso de las coordenadas para la tabla de errores) y reacciona al primer carácter significativo para determinar el camino de transición del estado.

Validación Estricta de Cadenas Literales: Se implementó un mecanismo de seguridad fundamental dentro del escáner de cadenas (textos entre comillas). En lugar de aceptar cualquier carácter arbitrario, el analizador verifica cíclicamente que el contenido interior solo posea caracteres alfanuméricos y de puntuación estándar (incluyendo caracteres hispanos por su valor ASCII extendido). Cualquier carácter no estándar detectado dentro de una cadena (ej. @, #, |) forzará al autómata a clasificar el bloque completo como un ERROR_LEXICO. Esto protege las fases posteriores de inyecciones de caracteres inválidos.



5. Gramática Libre de Contexto (GLC)

El lenguaje obedece a la siguiente gramática formal LL(1), la cual fue adaptada para ser procesada sin ambigüedades por el analizador sintáctico:

```
<programa>   ::= TABLERO CADENA "{" <columnas> "}"
<columnas>   ::= <columna> <columnas> | <columna>
<columna>    ::= COLUMNA CADENA "{" <tareas> "}" ";"
<tareas>     ::= <tarea> "," <tareas> | <tarea>
<tarea>      ::= tarea ":" CADENA "[" <atributos> "]"
<atributos>  ::= <atributo> "," <atributos> | <atributo>
<atributo>   ::= prioridad ":" <prioridad>
               | responsable ":" CADENA
               | fecha_limite ":" FECHA
<prioridad>  ::= ALTA | MEDIA | BAJA
```

6. Análisis Sintáctico y Árbol de Derivación
   La gramática anterior se implementó mediante un Parser Descendente Recursivo (Top-Down Parser). Cada regla de producción no terminal (como o) fue traducida directamente a un método en C++ dentro de la clase SyntaxAnalyzer.

   El parser solicita el token actual (peek()) e invoca métodos de validación (match() y consume()) para avanzar en la lectura. Conforme las reglas gramaticales son validadas positivamente, los métodos instancian punteros de NodoArbol, vinculando jerárquicamente las derivaciones para construir el árbol en memoria.

   Para las reglas que permiten recursividad por la derecha (como las listas de atributos separadas por comas), se emplearon bucles iterativos optimizados para evitar el desbordamiento de la pila de llamadas (stack overflow) y manejar de forma segura la presencia o ausencia de la coma final en las declaraciones.

7. Manejo y Estrategia de Recuperación de Errores
   El sistema implementa una gestión de errores asimétrica dependiendo de la fase de compilación en la que se detecte la anomalía, priorizando la estabilidad del sistema:

   1. Fase Léxica (Tolerancia a fallos): Cuando el autómata encuentra un carácter o secuencia inválida, registra el error y su coordenada en el ErrorManager. Para evitar un efecto dominó (donde la ausencia de un token rompe toda la estructura sintáctica), el sistema encapsula el error bajo un "disfraz" léxico y lo. transfiere al parser. Esto permite que el análisis continúe y se puedan reportar múltiples errores en una sola pasada.
   2. Fase Sintáctica (Panic Mode Estricto): Dada la naturaleza estructurada del TaskScript, los errores sintácticos indican una falla grave en la jerarquía del

documento. Al detectar un token que viola la gramática, el parser invoca una excepción (std::runtime_error), interrumpiendo inmediatamente la construcción del AST. Esta política asegura que el motor nunca intente recorrer o evaluar un árbol de memoria corrupto o incompleto, bloqueando por seguridad la generación final de reportes visuales.

8. Generación del Grafo y Reportes HTML Al finalizar un análisis sintáctico exitoso, la clase ReportGenerator toma control del flujo. El módulo extrae los nodos del árbol mediante algoritmos recursivos, aislando la data relevante (nombre de las tareas, responsables y prioridades). Para la visualización estructural, se genera código estandarizado en formato DOT. Para la presentación de datos, el sistema inyecta la información recolectada directamente en streams de salida (utilizando std::ofstream), generando archivos .html independientes con estilos CSS estáticos embebidos que garantizan su visualización offline en cualquier entorno web.

9. Justificación de Decisiones de Diseño El diseño de TaskScript Pro se fundamentó en la eficiencia, el desacoplamiento de componentes y el cumplimiento estricto de las restricciones académicas. A continuación se detallan y justifican las decisiones arquitectónicas clave:

1. **Arquitectura Desacoplada (React + C++ con Sockets Nativos)** Aunque el enunciado sugería el uso de frameworks nativos como Qt o wxWidgets para la interfaz gráfica, se optó (con previa autorización del auxiliar) por una arquitectura ClienteServidor. El frontend se desarrolló en React para proveer una interfaz de usuario asíncrona, moderna y amigable. El backend se programó en C++ puro utilizando la librería nativa para exponer un servidor local. Justificación: Esta separación permite que el análisis del Árbol de Sintaxis Abstracta (AST) y la manipulación de punteros en memoria ocurran de forma aislada y veloz en C++, mientras que React se encarga exclusivamente de la renderización del DOM y la presentación de los reportes. Esto evita

bloquear el hilo principal de la interfaz durante el escaneo de archivos grandes.

2. **Autómata Léxico Manual vs. Expresiones Regulares**
   Cumpliendo con la restricción del proyecto de no utilizar std::regex ni herramientas automáticas, el análisis léxico se programó utilizando lectura bajo demanda de caracteres. Justificación: Evaluar carácter por carácter en un flujo continuo otorga un control absoluto sobre las coordenadas exactas (línea y columna) del cursor. Esto es vital para el mecanismo de reporte de errores y permite detectar anomalías en tiempo constante, sin la sobrecarga computacional que implicaría compilar expresiones regulares para cada token.

3. **Estrategia de Recuperación de Errores Asimétrica** Se implementaron dos enfoques distintos para el manejo de errores, balanceando la flexibilidad y la seguridad de la memoria:

   • Léxico (Tolerancia Activa): Si el lexer detecta símbolos ilegales dentro de una cadena, encapsula el error y provee un token enmascarado al parser. Esto permite que el sistema no deje "huecos" estructurales y pueda continuar escaneando para reportar múltiples errores en una sola pasada.

   • Sintáctico (Panic Mode Estricto): Un error sintáctico significa que la jerarquía estructural del archivo está rota. En lugar de intentar recuperar nodos del AST de forma ambigua, el parser lanza una excepción que aborta instantáneamente la construcción del árbol.

   • Justificación: Esta asimetría protege el puntero del nodo raíz. Evita que el generador de reportes intente recorrer memoria corrupta o punteros nulos, garantizando que las salidas visuales solo se generen si la integridad estructural es comprobable al 100%.

4. **Inyección de Reportes en HTML Embebido**
   En lugar de depender de librerías externas pesadas, la lógica de compilación en C++ utiliza flujos de salida (std::ofstream) para generar archivos HTML estáticos. Justificación: Al inyectar código CSS directamente dentro de las etiquetas

CONCLUSION:

La culminación de este proyecto permitió validar la eficiencia de la arquitectura desacoplada entre un motor de análisis robusto en C++ y una interfaz moderna en React. La implementación exitosa de un Analizador Sintáctico Descendente y la gestión de un Árbol de Sintaxis Abstracta (AST) demostraron ser fundamentales para transformar un lenguaje de dominio específico (TaskScript) en herramientas visuales útiles como el tablero Kanban. Asimismo, la integración de una estrategia de recuperación de errores por Modo Pánico garantizó la resiliencia del software, permitiendo la generación de reportes informativos incluso ante entradas con errores sintácticos.